



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Time Series Analysis and Basic Image Processing

EIN087B - Ciencia de Datos
Paralelo 701

Profesor: Jorge Portilla G.
`jorge.portilla@usm.cl`

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Series de tiempo

- ▶ Presentar un nuevo ámbito de problema basado en datos con presencia de una dimensión temporal.
- ▶ Explicar estrategias generales para el análisis de series de tiempo.
- ▶ Identificar la descomposición básica de las series de tiempo.
- ▶ Introducir la aplicación de Ciencia de Datos para el procesamiento de series de tiempo.

Hemos visto distintos ejemplos a lo largo de curso. Sin embargo, tienen en común que las observaciones no tienen orden ni temporalidad.

- ▶ En la mayoría de problemas reales, donde lo que va a pasar con una variable y depende no sólo de algunas variables independientes x , sino también depende de lo que ha pasado antes con la propia variable y .
- ▶ Nuestros modelos, por ahora, no consideran para nada la historia de y al momento de realizar su regresión o clasificación.

¿Qué es una Serie de Tiempo?

$$\{y_t\}_{t=1}^T = y_1, y_2, y_3, \dots, y_T$$

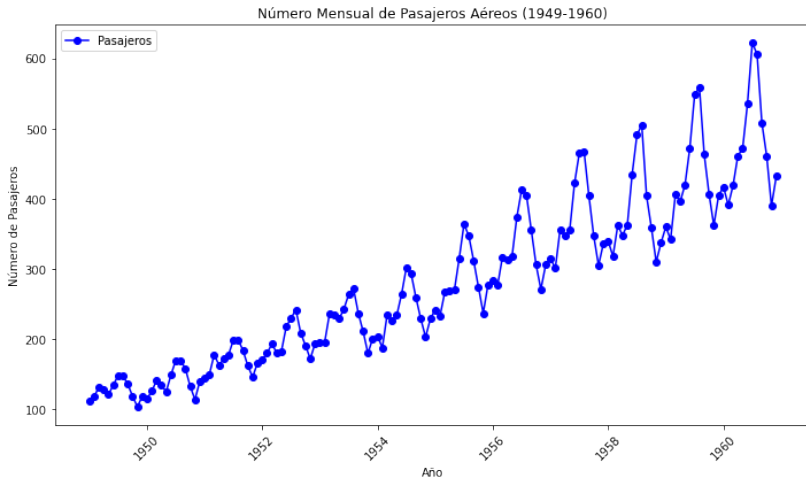
- ▶ Una **serie de tiempo** es una secuencia de observaciones ordenadas en el tiempo.
- ▶ Cada observación y_t se asocia a un instante t específico.
- ▶ Permite estudiar la evolución de una variable a lo largo del tiempo.

Ejemplos de aplicación:

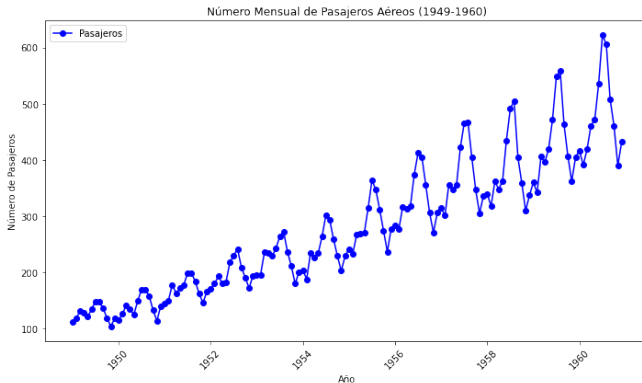
- ▶ **Climatología:** temperatura diaria, precipitaciones.
- ▶ **Finanzas:** precios de acciones, tasas de interés.
- ▶ **Marketing:** ventas mensuales, demanda de productos.
- ▶ **Ingeniería:** monitoreo de sensores industriales.
- ▶ **Salud:** ritmo cardíaco, evolución de epidemias.

Número mensual de pasajeros aéreos

Un set de datos clásico en cursos sobre Series de Tiempo corresponde a la medición del número mensual de pasajeros aéreos en miles desde enero de 1949 hasta diciembre de 1960, que está disponible en statsmodels.



Número mensual de pasajeros aéreos



- Para una línea aérea predecir con anticipación el volumen de pasajeros que tendrá que atender el próximo mes le ayudaría mucho a organizar su operación.
- Corresponde entonces a un problema de regresión donde la cantidad mensual de pasajeros es la variable dependiente y , pero ¿dónde están las variables independientes x ?

- ▶ ¿Como encontrar patrones en las series temporales?
- ▶ Los métodos de Machine Learning (ML) y Deep Learning (DL) suelen presentarse como soluciones avanzadas para problemas de predicción.
- ▶ **Métodos clásicos** como **Medias Móviles (MA)**, **Modelos Autoregresivos (AR)**, y **ARIMA** superan a modelos complejos (Árboles de Decisión, MLP, LSTM). Esto se observa tanto en:
 - **Pronóstico a un paso**
 - **Pronóstico a múltiples pasos**

- ▶ Un enfoque simple para pronóstico consiste en asumir que el valor futuro será igual al último valor observado:

$$\hat{y}_{t+1} = y_t$$

- ▶ Es barato computacionalmente, pero inexacto ante cambios bruscos o estacionales.
- ▶ Una mejora natural es promediar varios valores recientes en lugar de usar uno solo.
- ▶ Así surge el concepto de **Media Móvil (MA)**.

- ▶ Método simple e intuitivo para suavizar y pronosticar series temporales.
- ▶ Se basa en calcular el promedio de los últimos n valores anteriores a t :

$$\hat{y}_t = \frac{1}{n} \sum_{i=1}^n y_{t-i}$$

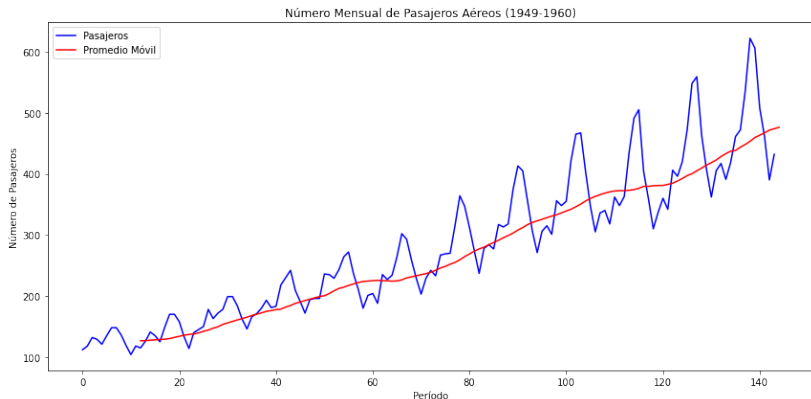
- ▶ La ventana de tamaño n se desplaza a lo largo del tiempo.
- ▶ Útil para reducir ruido y detectar tendencias locales.

Nota: Este modelo no considera estacionalidad ni tendencias a largo plazo.

Se debe definir el tamaño de la ventana que se desliza sobre la serie temporal.

Media Móvil (Moving Average)

Window size = 12



- Elegir una ventana de tamaño 12 no es arbitrario: refleja la estructura anual (12 meses).
- Con 144 valores observados, solo se pueden calcular 132 medias móviles reales.

- Sin embargo, el promedio móvil también permite predecir el valor del mes 145:

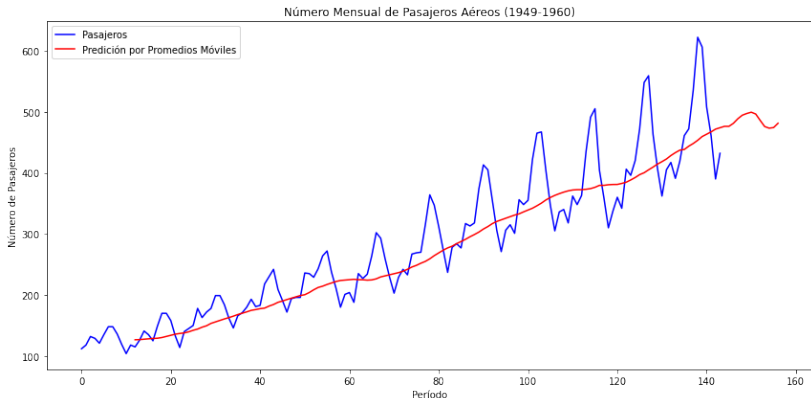
$$\hat{y}_{145} = \frac{1}{12} \sum_{i=1}^{12} y_{144-i+1}$$

- Para predecir el mes 146, no tenemos y_{145} , pero podemos usar $\hat{y}_{145}$ en su lugar.
- Así, podemos generar predicciones extendidas por varios meses usando:

$$\hat{y}_t = \frac{1}{12} \sum_{i=1}^{12} \tilde{y}_{t-i}$$

donde $\tilde{y}_{t-i}$ puede ser una observación real o predicha.

- Esto permite extender el horizonte de predicción, por ejemplo, hasta 12 meses futuros.



- ▶ A medida que extendemos la predicción, usamos menos datos reales y más valores predichos.
- ▶ El valor número 13 en el horizonte futuro es promedio de 12 valores que ya son promedios móviles.

- ▶ Para mejorar estas predicciones se proponen:
 - **Medias Móviles Ponderadas (WMA)**: asignan pesos distintos a cada punto de la ventana.
 - **Suavizado Exponencial**: aplica pesos que decrecen exponencialmente, dando mayor relevancia a los datos recientes.
- ▶ A pesar de sus mejoras, estos métodos siguen siendo predictores de corto plazo.
- ▶ **Lo destacable: las medias móviles capturan la tendencia central de la serie.**
- ▶ Esta observación motiva estrategias más robustas para el modelado de series de tiempo.

Descomposición de Series de Tiempo

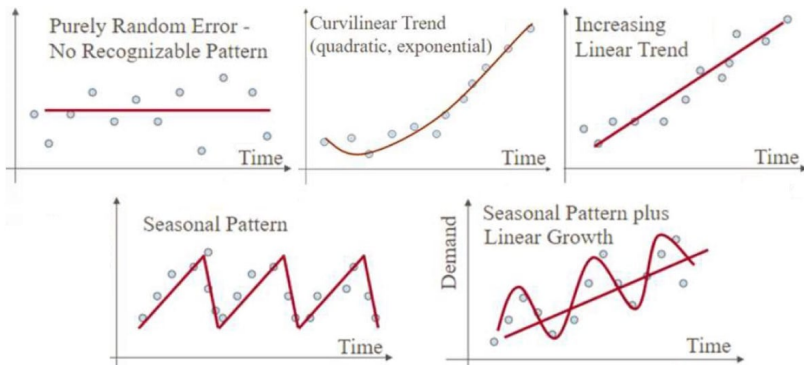
- Una serie de tiempo puede entenderse como la suma (o producto) de varios componentes subyacentes.

Descomposición de Series de Tiempo

- Una serie de tiempo puede entenderse como la suma (o producto) de varios componentes subyacentes.
 - **Tendencia (T_t)**: Evolución a largo plazo (crecimiento, caída, etc.).
 - **Estacionalidad (S_t)**: Patrones que se repiten en intervalos regulares (ej. mensual, anual).
 - **Residuo (R_t)**: Variaciones no explicadas por los componentes anteriores.

Descomposición de Series de Tiempo

- Una serie de tiempo puede entenderse como la suma (o producto) de varios componentes subyacentes.
 - **Tendencia** (T_t): Evolución a largo plazo (crecimiento, caída, etc.).
 - **Estacionalidad** (S_t): Patrones que se repiten en intervalos regulares (ej. mensual, anual).
 - **Residuo** (R_t): Variaciones no explicadas por los componentes anteriores.

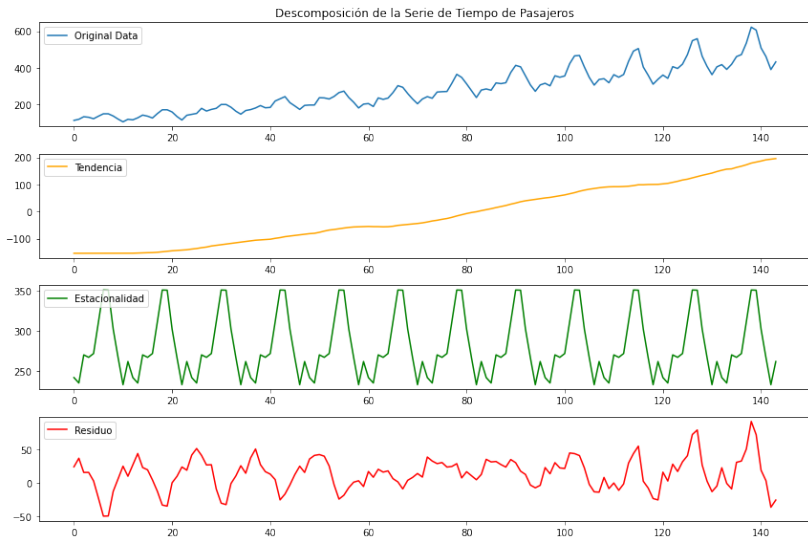


- Ejemplo de descomposición aditiva:

$$y_t = T_t + S_t + R_t$$

- También puede existir una **Componente Cíclica (C_t)** que refleja oscilaciones irregulares de largo plazo.
- **Estacionalidad:**
 - Patrón que se repite en intervalos regulares.
 - Ejemplos: ventas de fin de año, temperatura estacional.
 - Predecible a partir de la variable temporal t .
- **Ciclos:**
 - Fluctuaciones sin periodicidad fija.
 - Ejemplos: ciclos económicos, fenómenos políticos.
 - No predecibles únicamente con t .
 - No se modelan directamente, suelen quedar en el residuo.
- **Importancia:** Distinguir entre estacionalidad y ciclos evita errores de interpretación y modelado.

Modelo Aditivo de Descomposición (tendencia, estacionalidad y residuo)



Cálculo de Componentes del Modelo Aditivo

Dado que la serie se modela como:

$$y_t = T_t + S_t + R_t$$

calculamos cada componente de forma manual:

1. **Estacionalidad** (S_t): Promedio de **cada mes** a lo largo de los años:

$$S_t = \frac{1}{N} \sum_{j=1}^N y_{t+12(j-1)}$$

donde $t = 1, \dots, 12$ y N es el número de años.

2. **Tendencia** (T_t): Promedio móvil sobre los datos sin estacionalidad:

$$T_t = \frac{1}{k} \sum_{i=0}^{k-1} (y_{t+i} - S_{t+i})$$

con k el tamaño de la ventana móvil (ej: $k = 12$).

3. **Residuo** (R_t): Lo que queda al remover tendencia y estacionalidad:

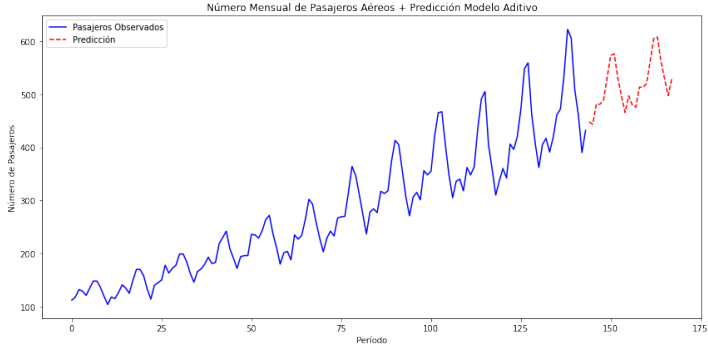
$$R_t = y_t - T_t - S_t$$

Este procedimiento permite analizar separadamente patrones a largo plazo, estacionales y aleatorios.

Predicción usando el modelo aditivo

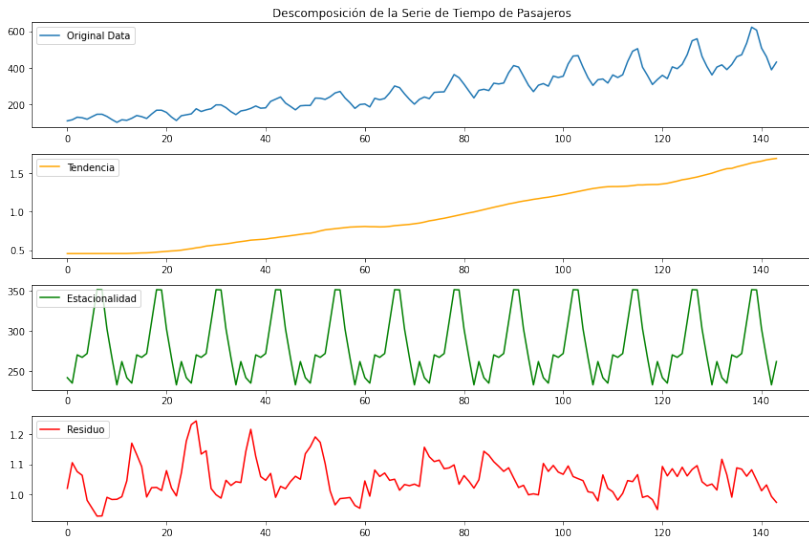
- Para predecir los próximos 24 meses, proyectamos la tendencia mediante regresión lineal (u otra técnica de extrapolación) y le sumamos la **estacionalidad** correspondiente. Así obtenemos la predicción final.

```
1 # Regresión lineal para calcular la tendencia futura
2 x = np.arange(len(trend) - (window_size - 1))
3 slope, intercept = np.polyfit(x, trend[window_size-1:], 1)
4 predictions = [] # Predicciones para los próximos 24 meses
5 for i in range(len(data), len(data) + 24):
6     trend_projection = slope * i + intercept
7     predictions.append(trend_projection + monthly_avg[i % 12])
```



Modelo Multiplicativo de Descomposición

Consiste en suponer que los datos observados son el resultado de la multiplicación de tres componentes: tendencia, estacionalidad y residual.



Cálculo de Componentes del Modelo Multiplicativo

Modelamos la serie como:

$$y_t = T_t \times S_t \times R_t$$

Pasos del cálculo manual:

1. Estacionalidad (S_t):

- ▶ Se calcula como el promedio de cada mes (enero, febrero, ...) a lo largo de los años.
- ▶ Resulta en una lista de 12 valores que se repite a lo largo de toda la serie.

2. Tendencia (T_t):

- ▶ Se remueve la estacionalidad dividiendo la serie: y_t/S_t
- ▶ Luego se calcula un promedio móvil sobre la serie desestacionalizada.

3. Residuo (R_t):

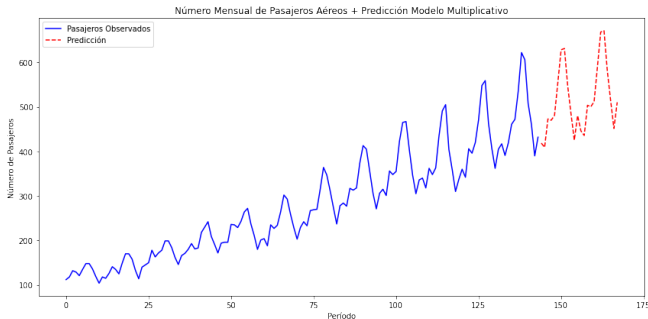
- ▶ Es la parte no explicada por tendencia ni estacionalidad:

$$R_t = \frac{y_t}{T_t \times S_t}$$

Este modelo es útil cuando la estacionalidad crece proporcionalmente con la tendencia.

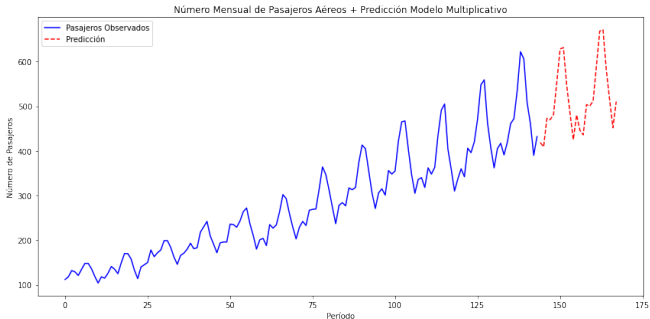
Modelo Multiplicativo de Descomposición

```
1 x = np.arange(len(trend) - (window_size - 1))
2 slope, intercept = np.polyfit(x, trend[window_size-1:], 1)
3
4 predictions = [] # Predicciones para los pr ximos 24 meses
5 for i in range(len(data), len(data) + 24):
6     trend_projection = slope * i + intercept
7     # Estacionalidad: repetimos la estacionalidad conocida
8     seasonality_projection = monthly_avg[i % 12]
9     # En el modelo multiplicativo, la predicción es la tendencia *
10     estacionalidad
11     prediction = trend_projection * seasonality_projection
12     predictions.append(prediction)
```



Modelo Multiplicativo de Descomposición

- El modelo multiplicativo logra capturar el incremento en la magnitud de la estacionalidad.
- Genera predicciones que coinciden con la intuición, ofreciendo mayor confianza en los resultados.
- La estacionalidad se ve amplificada por la tendencia en este tipo de modelo.
- Es especialmente adecuado para series temporales donde las variaciones estacionales no son constantes, sino que aumentan proporcionalmente a la tendencia.



Modelos Autorregresivos

Modelos Autorregresivos (AR)

- ▶ Los **modelos autorregresivos (AR)** son modelos de series temporales donde el valor actual depende linealmente de sus valores pasados.
- ▶ Un modelo autorregresivo de orden p , denotado **AR(p)**, usa los últimos p valores para predecir el valor actual.

Ecuación general:

$$\hat{y}_t = \phi_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p}$$

Donde:

- ▶ $\hat{y}_t$: valor predicho en el tiempo t
- ▶ $y_{t-1}, \dots, y_{t-p}$: valores pasados (rezagos)
- ▶ $\phi_0, \dots, \phi_p$: parámetros del modelo

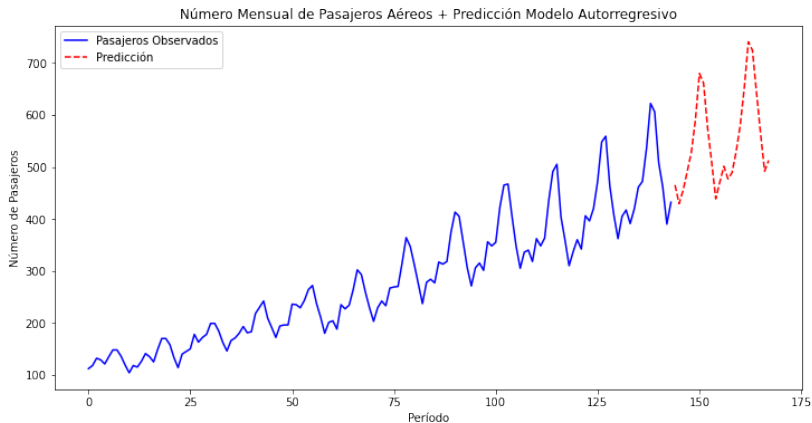
Implementación práctica:

- ▶ Puede implementarse con `LinearRegression` de `sklearn`
- ▶ Ya se ha utilizado en el curso de Ciencia de Datos

Modelo Autorregresivos (AR)

```
1 # Definiremos una función para transformar los datos para el modelo
  AR
2 # consiste en construir vectores X usando valores y rezagados.
3 def transform_data(data, lags):
4     X, y = [], []
5     for i in range(lags, len(data)):
6         X.append(data[i-lags:i])
7         y.append(data[i])
8     return np.array(X), np.array(y)
9
10 # Definimos un número de lags (retardos) para el modelo AR.
    Siguiendo la lógica de la estacionalidad anual, definamos 12.
11 lags = 12
12 X, y = transform_data(data, lags)
13
14 model = LinearRegression()
15 model.fit(X, y)
16
17 # Vamos a predecir un paso adelante en cada iteración
18 predictions = []
19 input_data = list(data[-lags:])
20 for i in range(24): # predecir 24 meses hacia adelante
21     prediction = model.predict([input_data[-lags:]])
22     predictions.append(prediction[0])
23     input_data.append(prediction[0])
```

Modelo Autorregresivos (AR)



- ▶ En cada paso de predicción, el vector de entrada x_t se forma con los valores pasados $[y_{t-n}, \dots, y_{t-1}]$.
- ▶ Cada valor es multiplicado por un peso ϕ y se suma un intercepto ϕ_0 :

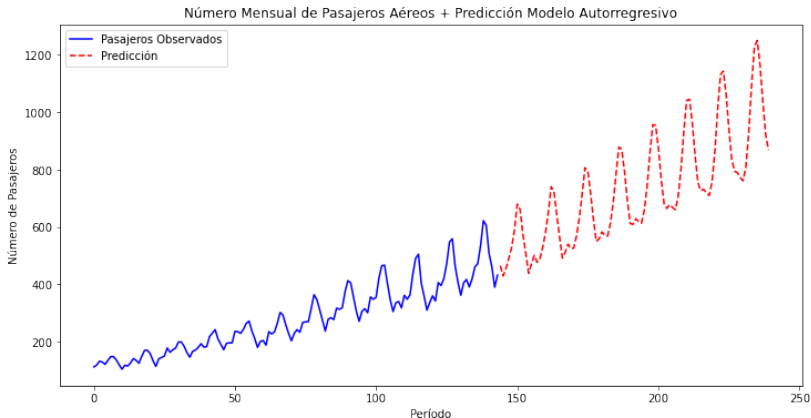
$$\hat{y}_t = \phi_0 + \sum_{i=1}^n \phi_i y_{t-i}$$

- ▶ Se parece a un promedio móvil ponderado, pero:
 - Los pesos ϕ_i son aprendidos para minimizar el error cuadrático medio (MSE).
 - No es necesario que sumen uno.
 - Incluye un intercepto ϕ_0 .

- ▶ El modelo AR asume que la serie es **estacionaria**.
- ▶ Una serie es estacionaria si:
 - Su media y varianza son constantes en el tiempo (estacionaria).
 - No muestra tendencia ni estacionalidad persistente.
- ▶ La serie de pasajeros parece ajustarse bien a corto plazo, pero:
 - No se ha evaluado cuantitativamente el modelo aún.
 - A largo plazo, la tendencia real no queda bien representada por el modelo AR.

Modelo Autorregresivo

En el caso anterior, funciona bien al proyectar 2 años. Si extendemos la proyección con el modelo AR, la tendencia de los datos originales no queda bien representada.



- ▶ La **diferenciación** es una técnica que transforma una serie no estacionaria en estacionaria.
- ▶ En lugar de analizar los valores absolutos de una serie, se analizan sus **variaciones** entre periodos consecutivos.

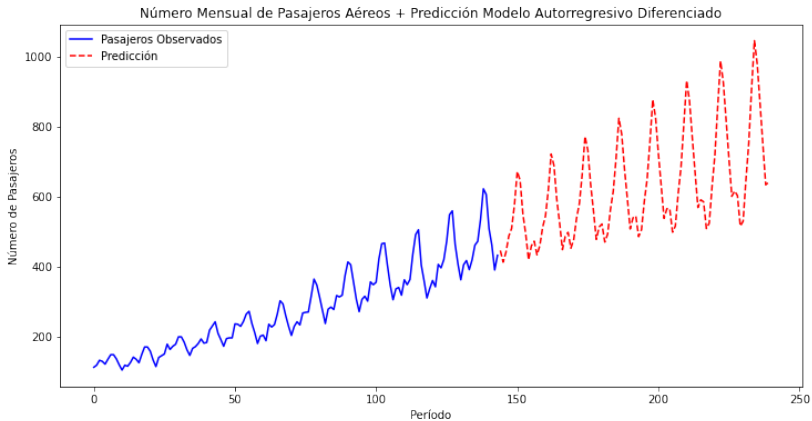
$$\Delta Y_t = Y_t - Y_{t-1} \quad (1)$$

- ▶ Esta transformación permite mitigar el efecto de tendencias o estacionalidades.
- ▶ Es posible aplicar la diferenciación más de una vez: el **grado de diferenciación** indica cuántas veces se ha realizado este proceso.
- ▶ Una vez alcanzada la estacionariedad, se puede aplicar un **modelo autorregresivo** sobre la serie diferenciada.
- ▶ Esto permite proyectar diferencias mensuales, y a partir de ellas, estimar niveles futuros de la serie original.

Diferenciación

```
1 differenced_data = [data[i]-data[i-1] for i in range(1, len(data))]
2 def transform_data(data, lags):# Transformar datos para AR
3     X, y = [], []
4     for i in range(lags, len(data)):
5         X.append(data[i-lags:i])
6         y.append(data[i])
7     return np.array(X), np.array(y)
8
9 X, y = transform_data(differenced_data, 12) # lags = 12
10 model = LinearRegression()# Modelo AR con datos diferenciados
11 model.fit(X, y)
12
13 predictions_diff = []# Datos diferenciados y recuperaci n de escala
14 input_data = list(differenced_data[-lags:])
15 pred_num = 96
16 last_original_value = data[-1]
17 predictions = []
18
19 for i in range(pred_num):
20     prediction_diff = model.predict([input_data[-lags:]])
21     predictions_diff.append(prediction_diff[0])
22
23     new_value = last_original_value + prediction_diff #Escala
24     predictions.append(new_value[0])
25     last_original_value = new_value input_data.append(
        prediction_diff[0])
```

Modelo Autorregresivo Diferenciado

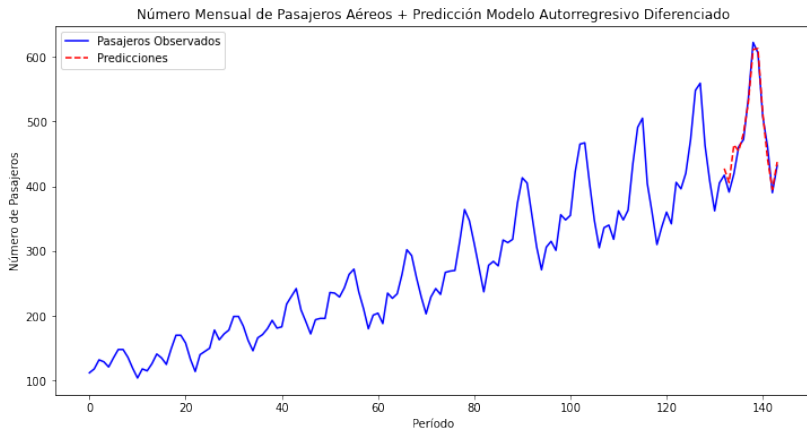


- ▶ Evaluar visualmente si la proyección "sigue la tendencia" no es una medida rigurosa del desempeño del modelo.
- ▶ A diferencia de datos no temporales, no podemos usar muestreo aleatorio: debemos respetar el orden cronológico.
- ▶ Por eso, se reserva el segmento final de la serie como **conjunto de prueba**, para evaluar la capacidad predictiva del modelo.
- ▶ En nuestro caso: entrenamos con los primeros 132 meses y predecimos los últimos 12 meses.
- ▶ Para cuantificar el error, usamos la **Raíz del Error Cuadrático Medio (RMSE)**, que mide el error promedio en unidades originales:

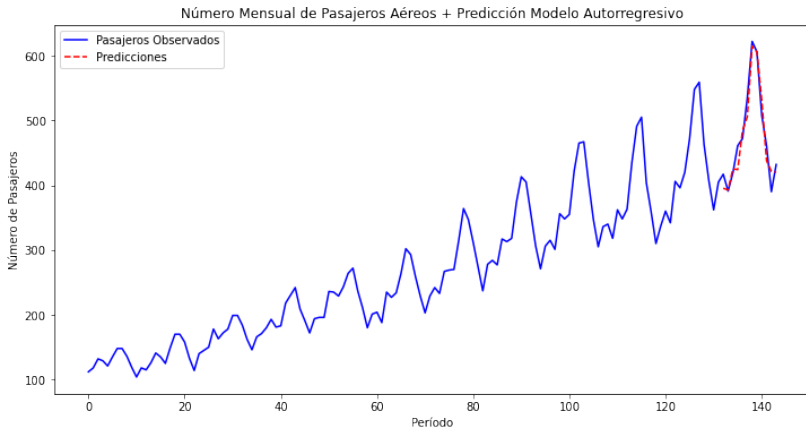
$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (\hat{Y}_t - Y_t)^2} \quad (2)$$

- ▶ Esta métrica penaliza errores grandes y permite comparar modelos de forma objetiva.

RMSE para el conjunto de prueba: 21.50



RMSE para el conjunto de prueba: 20.76



Apreciamos que el modelo autorregresivo sin diferenciar ajusta mejor a los datos de prueba, al menos en el corto plazo. El uso de diferenciación podría (suponemos) mejorar una estimación de largo plazo, pero no tenemos datos suficientes para validar a mucho más de un año.

- ▶ Existen múltiples modelos estadísticos diseñados específicamente para series temporales:
 - **ARIMA**: combina autorregresión, integración (diferenciación) y media móvil.
 - **SARIMA**: extiende ARIMA incorporando estacionalidades explícitas.
 - **SARIMAX**: incluye variables exógenas¹.
 - **Prophet (desarrollado por Facebook)**: modelo aditivo que incorpora múltiples estacionalidades y eventos especiales.
- ▶ Estos modelos aún se consideran predominantemente **estadísticos**, aunque algunos están en la frontera del **Machine Learning (ML)**, al requerir entrenamiento de parámetros.
- ▶ El foco de esta asignatura está en avanzar hacia modelos de ML, con mayor capacidad de generalización.
- ▶ Por eso, dejamos sin explorar muchos modelos especializados, para concentrarnos en lo que pueden aprender los modelos de ML cuando se alimentan con series temporales.

¹variable que no forma parte del proceso generador de la serie que queremos modelar, pero que puede influir en su comportamiento y ayudarnos a mejorar la predicción. Algunos ejemplos: el precio del combustible, vacaciones escolares, PIB mensual, eventos deportivos o festividades.

Machine Learning para Series de Tiempo

- ▶ Para aplicar modelos de aprendizaje supervisado clásico (como **KNN**, **SVM**, **Random Forest**) a series temporales, es necesario transformar el problema.
- ▶ Reformulamos la serie como un conjunto de ejemplos de entrada/salida:
 - Entrada: valores anteriores de la serie ($y_{t-1}, y_{t-2}, \dots$)
 - Salida: valor siguiente (y_t)
- ▶ Este proceso se conoce como **ventana deslizante**, y convierte el problema en una tarea clásica de predicción.

Ejemplo: si usamos una ventana de 12 meses, el modelo aprenderá a predecir el mes 13 usando los meses 1 al 12 como entrada.

- ▶ Aunque estos modelos no modelan directamente la dinámica temporal, pueden capturar patrones locales si se entrena correctamente.
- ▶ En esta etapa aplicaremos un modelo clásico de ML sobre los datos de **AirPassengers**.

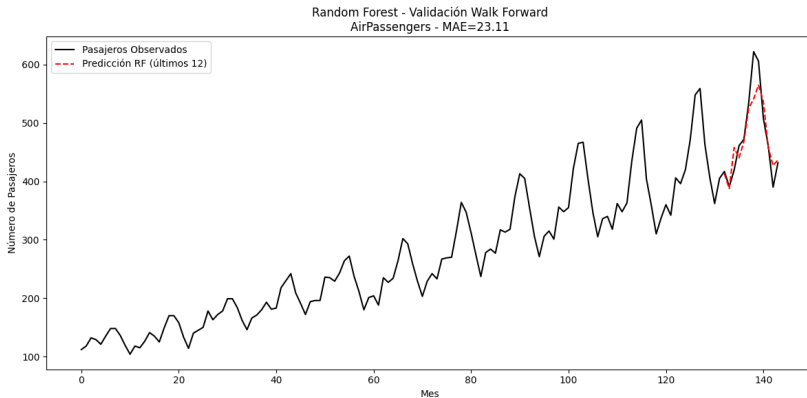
Random Forest con Validación Walk-Forward

- ▶ **1. Transformación supervisada:** usamos una ventana deslizante de 12 meses para predecir el mes siguiente.
- ▶ **2. Walk-forward validation:** simulamos un entorno real:
 - Entrenamos el modelo con todos los datos disponibles hasta el mes actual.
 - Predecimos el siguiente mes.
 - Añadimos el dato real a la historia y repetimos.
- ▶ Este enfoque permite evaluar el modelo en condiciones que reflejan cómo funcionaría en producción.
- ▶ Random Forest no modela la secuencia temporal directamente, pero aprende relaciones entre valores pasados y futuros.

Ventajas:

- ▶ Evita fuga de información (data leakage).
- ▶ Permite evaluación paso a paso del rendimiento del modelo.
- ▶ Mejora la validez externa de las predicciones.

Random Forest



- ▶ Los modelos clásicos como **Random Forest**, **KNN** o **Regresión Lineal** requieren transformar la serie temporal en ventanas estáticas.
- ▶ Aunque pueden capturar patrones locales, no modelan directamente la secuencia ni retienen memoria del pasado.

Alternativas para modelar secuencias temporalmente dependientes...

- ▶ **LSTM (Long Short-Term Memory)**: una red neuronal recurrente diseñada para aprender dependencias a largo plazo.
- ▶ **GRU (Gated Recurrent Unit)**: variante más simple y eficiente que LSTM.
- ▶ **CNNs temporales**: detectan patrones locales de forma eficiente.
- ▶ **Transformers para series de tiempo**: modelan relaciones globales sin recurrencia.

- ▶ Capacidad de generalizar patrones complejos no visibles para modelos lineales.
- ▶ Modelado directo del tiempo como parte del aprendizaje.
- ▶ Adaptabilidad a múltiples frecuencias, estacionalidades y ruido.

Introducción a Procesamiento de Imágenes

Desde Escala de Grises hasta Hiperespectral: Tipos de Imágenes en CV



- **Escala de Grises:** Imágenes compuestas únicamente por niveles de gris, representando la intensidad luminosa sin color.
- **RGB:** Imágenes a color que combinan los canales rojo, verde y azul, siendo las más comunes en visión por computadora.
- **Imágenes hiperespectrales** (con más de 100 bandas) e **imágenes multiespectrales** (con más de 10 bandas)².

²S. Zheng, W. Cai, C. Zhao, D. Li, J. Zhang, Q. Lu, On the measurement of flame temperature and emissivity based on multispectral imaging technique, Measurement (ISSN: 0263-2241) 196 (2022) 111272.

Multispectral imaging

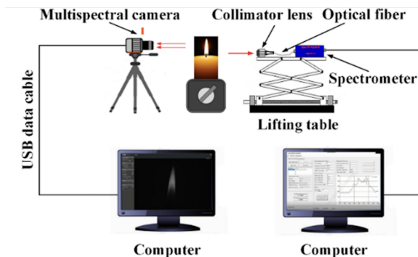


Fig. 7 Schematic of the candle flame detection system.

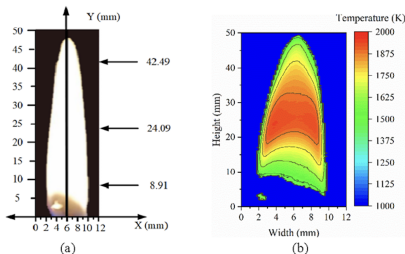


Fig. 8 (a) Image measured by multispectral imaging equipment. (b) Radiation temperature image of the thin-wick paraffin flame.

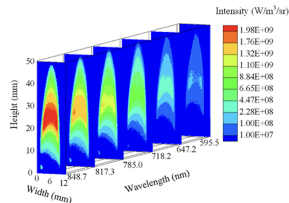


Fig. 9 The multispectral radiation intensity of the thin-wick paraffin flame.

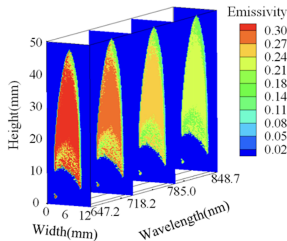


Fig. 10 Wavelength-dependent emissivity distribution of thin-wick paraffin flame. Including 4 images: $\lambda_1 = 647.2 \text{ nm}$, $\lambda_2 = 718.2 \text{ nm}$, $\lambda_3 = 785.0 \text{ nm}$, $\lambda_4 = 848.7 \text{ nm}$.

³Zheng, S., Cai, W., Zhao, C., Li, D., Zhang, J., Lu, Q. (2022). On the measurement of flame temperature and emissivity based on multispectral imaging technique. Measurement, 196, 111272.

Hyperspectral imaging

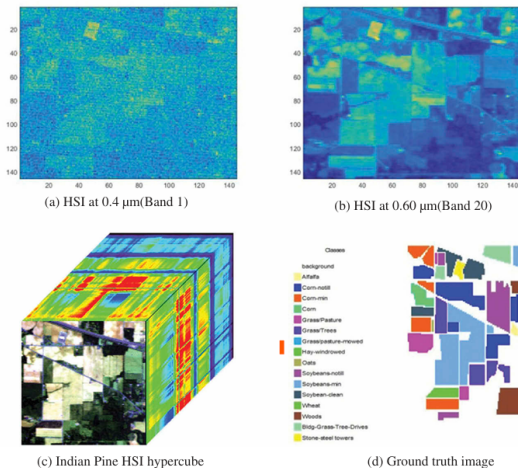
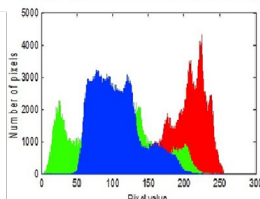
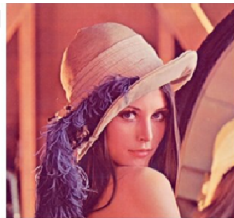
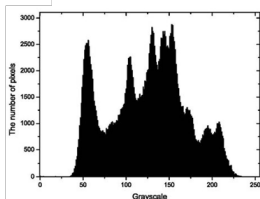
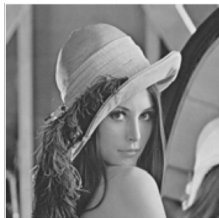


Figure 4. Indian Pines HSI dataset. The image (a) is the 1st band image of the Indian Pines HSI which is captured at 0.40 μm wavelength, the image (b) is the 20th band image of the HSI which is captured at 0.60 μm wavelength, the image (c) is the HSI hypercube of total 220 bands and the image (d) is a sample colour image with the thematic mapping and ground cover classes.

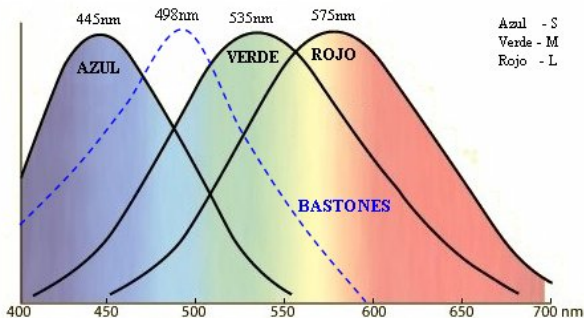
⁴Uddin, M. P., Mamun, M. A., Hossain, M. A. (2019). Effective feature extraction through segmentation-based folded-PCA for hyperspectral image classification. International Journal of Remote Sensing, 40(18), 7190-7220.

Escala de Grises VS Color:

- ▶ Cada punto representado por un pixel (picture element).
- ▶ Escala de Grises: 1 canal = 8 bits por pixel (bpp)
- ▶ Color: 3 canales = 24 bpp
- ▶ Color + transparencias: 4 canales = 32 bpp



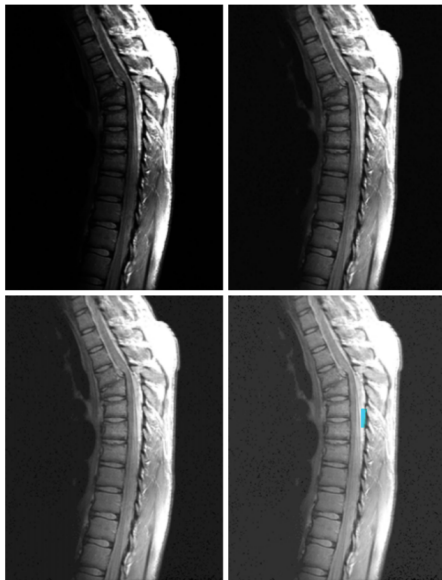
- **RGB:** Acrónimo de Red, Green, Blue.
- Dadas las características de absorción de la visión humana, los colores son percibidos como combinaciones variables de los colores conocidos como **primarios**: rojo (R), verde (G) y azul (B).



5

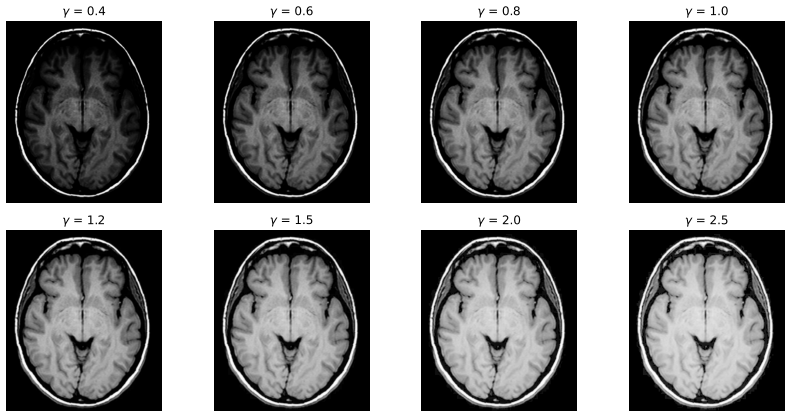
⁵ Los bastones son células fotorreceptoras en la retina del ojo, altamente sensibles a la luz, lo que les permite funcionar en condiciones de baja iluminación (visión nocturna o escotópica).

Magnetic resonance



- Imagen de resonancia magnética (RM) de una columna vertebral humana fracturada. Resultados de aplicar la transformación y $\gamma = 0,6, 0,4$, y $0,3$, respectivamente.
- (Imagen original del Dr. David R. Pickens, Departamento de Radiología y Ciencias Radiológicas, Centro Médico de la Universidad de Vanderbilt).

⁵Imagen tomada de <https://i.static.net/44vlo.png>



6

- ▶ Se aplican distintas transformaciones con valores de γ entre 0.5 y 3.0.
- ▶ Valores de $\gamma < 1$ aclaran la imagen.
- ▶ Valores de $\gamma > 1$ oscurecen la imagen.

- Histograma de una imagen:

$$h(k) = n_k$$

con $k \in \{0, \dots, L\}$ nivel de gris (para L niveles de gris), y n_k el número de píxeles de la imagen con valor de nivel de gris k .

- Es común también utilizar la versión normalizada del histograma en $[0; 1]$:

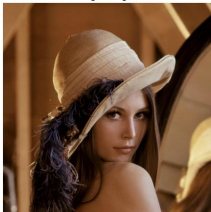
$$p(k) = \frac{n_k}{n}$$

con n como el número total de píxeles en la imagen.

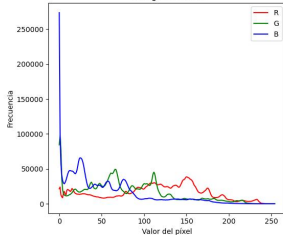
- Se puede decir que $p(k)$ es un estimador de la probabilidad de ocurrencia del nivel de gris k en la imagen.
- Histogramas son herramientas importantes para segmentación de imágenes en tiempo real (simples y de bajo costo computacional).

Histograma 2D

Imagen Original



Histogramas RGB



Histogramas RGB (Escala Log)

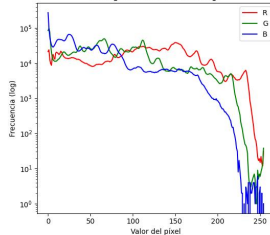
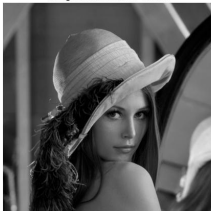
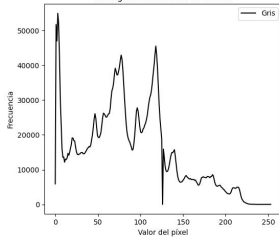


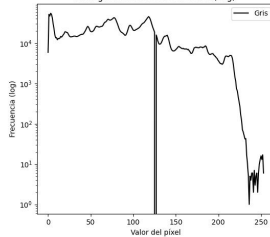
Imagen en Escala de Grises



Histograma en Escala de Grises



Histograma en Escala de Grises (Log)



Revisar enlace <https://docs.baslerweb.com/histogram>

Comparación de Histogramas

- **Distancia de Bhattacharyya**, una de las más usadas.
- Entrega valor normalizado en $[0, 1]$ de distancia para histogramas normalizados.

$$\text{► } D_B(h_1, h_2) = \sqrt{1 - \sum_{i=1}^N \sqrt{h_1^{(i)} \times h_2^{(i)}}}$$

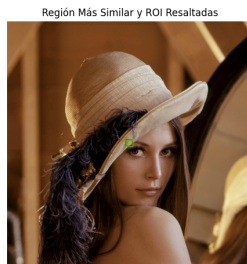
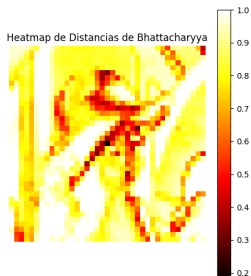
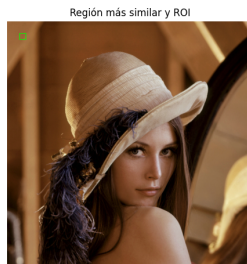
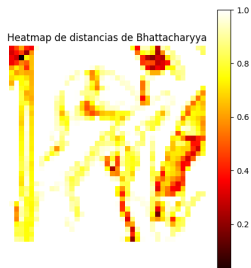
con N el número de bins de los histogramas y $h_j^{(i)}$ el i -ésimo bin del histograma j .

```
// The Bhattacharyya Distance , for normalized histograms

double BhattacharyyaDistance(float *h1, float *h2, int N) {
    double D = 0.0;
    for (int i = 0; i < N; i++) {
        D += sqrt(h1[i] * h2[i]);
    }
    return 1 - D < 0 ? 0.0 : sqrt(1 - D);
}
```

- Se puede usar para clasificar regiones, según objetos de interés.

Clasificar regiones

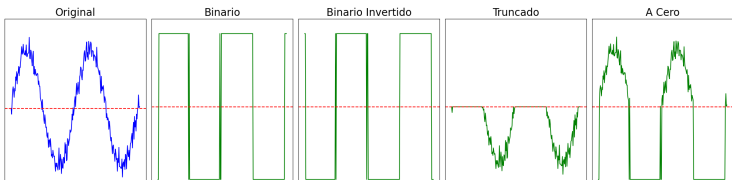


Particionar una imagen en múltiples segmentos (conjuntos de píxeles) [Shapiro and Stockman, 2001].

- Método simple **segmentación binaria**.
- Aplicaciones típicas: segmentación de movimiento, separar objeto de interés.
- Umbralización destaca píxeles de interés.

```
cv::threshold(src , dst , threshold , max_value , type );
```

- *src* y *dst* imágenes de un canal.
- *threshold* el umbral (línea azul).
- *max_value* valor de referencia a modificar (depende de método).
- *type* tipo de umbralización (0: Binario; 1: Binario Invertido; 2: Truncado; 3: Umbral A Cero; 4: Umbral A Cero Invertido).





UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Time Series Analysis and Basic Image Processing

EIN087B - Ciencia de Datos
Paralelo 701

Profesor: Jorge Portilla G.
`jorge.portilla@usm.cl`

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile